Password rules for Coup

How a player chooses a password, and what the strength bars mean

Version 1 · 26 October 2026 · Coup graduation project · Status: agreed proposal. The frontend checker follows these rules, and the register screen will use it; the backend will use the same rules when it is built.

1. Why we need password rules

A password protects a player's account, their Gold and Gems, and the skins they own. If someone guesses the password, they can take all of these.

The register screen will check the password while the player types. This gives quick help. But a browser check alone is not enough: a person can skip it with a tool such as Burp Suite and send a request straight to the server. So the server must check the same rules again. Both sides read one shared rules file, so the rules cannot drift apart.

2. The rules

Rule	What it means
At least 15 characters	Short passwords are easy to guess. Long passwords, such as four random words, are strong and easy to remember.
At most 128 characters	This stops very long inputs that could slow down the server.
Any character is allowed	Letters, numbers, symbols, spaces, and any language (for example Arabic) are all fine. Each character counts once.
No forced mix	We do **not** ask for a capital letter, a number, or a symbol.
Not a common password	Passwords from lists of common or leaked passwords are refused, also when only the capital letters are different.
Nothing personal	The password must not contain the player's name, the name part of their email (before the @), or the word "coup".
Strong enough	The strength bars must show at least **Fair**. A **Weak** password is refused.

When a password is refused, the player sees the reason, for example "Use at least 15 characters." or "This password is too common."

3. Why we chose these rules

- **15 characters.** NIST says that a password which is the only way to log in must have at least 15 characters. It allows 8 characters only when the password is used together with another factor.[1] Players log in with a password only, so we use 15.

- **Any character, counted once.** NIST says the system should accept all printable characters, the space, and Unicode letters. It should normalize the text the same way (Unicode NFC) and count each character once.[1]
- **No forced mix.** NIST says the system must not ask for a mix of character types.[1] Mix rules make people choose patterns like "Password1!", which attackers try early.
- **Blocklist.** NIST says the system must compare the whole new password with a list of common, expected, or leaked passwords, and must tell the user why it was refused.[1]
- **Personal words.** Attackers try a person's name and email first. The strength estimator also uses these words, so a password built from them gets a low score.[2][4]
- **No forced changes and no hints.** NIST says the system must not force regular password changes and must not store password hints. It must ask for a new password only when there is evidence that the old one was stolen.[1]
- **Strength bars.** NIST says the system must help users choose a strong password.[1] The strength bars give this help while the player types.

4. The strength bars

Below the password field there are four bars and a word. They come from **zxcvbn**, a well-known strength estimator from Dropbox.[2][4] It estimates how many guesses an attacker would need, and it gives a score from 0 to 4.[4] We show the score like this:

What the player sees	Score	Guesses an attacker needs[4]	Result
–	–	–	Before typing. The hint says "At least 15 characters".
Weak	0–1	fewer than 1 million	**Refused.** One red bar.
Fair	2	fewer than 100 million	Accepted. Two yellow bars. This is the minimum.
Good	3	fewer than 10 billion	Accepted. Three green bars.
Strong	4	10 billion or more	Accepted. Four light purple bars, the "legendary" color of the game.

> If any rule fails (too short, too long, common, or personal), the bars show **Weak**, even when the score is high. For example, an 11-character random password can get a high score, but it is still too short.

5. Examples

These examples come from the shared test file. Do not use them as real passwords.

Password	Bars	Why
`short password`	Weak, refused	Only 14 characters.
`password12345678`	Weak, refused	A common word plus a simple number sequence.
`1qaz2wsx3edc4rfv`	Weak, refused	A keyboard pattern that is on the common-password list.
`welcome to coup!`	Weak, refused	Contains the word "coup".
`football20262026`	Fair	Long enough, but built from a common word and a repeated year.
`sunshine summer1`	Good	Two words and a number.
`correct horse battery staple`	Strong	Four random words. Long and easy to remember.

6. Tips for players

- Use a passphrase: four or more random words, for example from a list or from things around you.
- Use a different password for every website. A password manager can remember them for you.
- Never share your password. The Coup team will never ask for it.

7. How we keep the browser and the server the same

- **One rules file:** `integration/contracts/password-policy.json`. The frontend checker reads it now; the server will read it later.
- **One test file:** `integration/mocks/password-cases.json` with 19 test passwords. The browser tests and the future server tests must give the same result for each one.
- **Same estimator:** the browser uses zxcvbn 4.4.2.[4] The server should use zxcvbn-python 4.5.0, a port of the same version, with `max_length=128`, because its default limit is 72 characters.[5] We ran all 19 test passwords through both versions, and the results were the same.
- **Safe storage:** the server must hash passwords with Argon2id (at least 19 MiB of memory, 2 iterations, and 1 degree of parallelism) and a unique salt for each password.[3] It must not use bcrypt, because bcrypt reads only the first 72 bytes of a password.[3]
- **Limited attempts:** the server must limit failed logins. NIST allows at most 100 failed attempts in a row; a lower limit is better.[1]
- **No leaks:** the server must never log a password or send it back in a response.

References

1. National Institute of Standards and Technology (NIST). *Digital Identity Guidelines: Authentication and Authenticator Management*, NIST SP 800-63B-4, 2025. https://pages.nist.gov/800-63-4/sp800-63b.html (accessed 26 October 2026).
2. D. L. Wheeler. "zxcvbn: Low-Budget Password Strength Estimation." In *Proceedings of the 25th USENIX Security Symposium*, Austin, TX, USA, August 2016, pp. 157–173. https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/wheeler
3. OWASP Foundation. "Password Storage Cheat Sheet." OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed 26 October 2026).
4. Dropbox. "zxcvbn: Low-Budget Password Strength Estimation," version 4.4.2. https://github.com/dropbox/zxcvbn
5. zxcvbn-python contributors. "zxcvbn-python," version 4.5.0. https://github.com/dwolfhub/zxcvbn-python